

Directing Measures and Markov Mixtures:

Formalizing Diaconis–Freedman 1980 in Lean 4

Mettapedia Project

April 2026

Abstract

We present a complete Lean 4 formalization of the Markov de Finetti theorem (Diaconis–Freedman 1980): every Markov-exchangeable, recurrent prefix measure on a finite alphabet is a mixture of time-homogeneous Markov chains. The formalization comprises 23,899 lines of Lean code with zero unproved assumptions (`sorry`), building on a 43,683-line formalization of the classical (i.i.d.) de Finetti theorem. Key innovations include a directing measure uniqueness lemma that transfers de Finetti structure across measure restrictions, and a carrier transport bijection that establishes start-state invariance via trajectory segment swapping. The proof architecture decomposes into successor-matrix partial exchangeability, per-row de Finetti factorization, and cross-row coherence induction. This formalization provides a verified foundation for structured universal mixtures in the style of Solomonoff induction, where Markov models replace i.i.d. hypotheses for tractable sequential prediction.

Contents

1	Introduction	2
1.1	The Theorem	2
1.2	Why Formalize?	3
1.3	Contribution	3
2	Mathematical Background	3
2.1	Prefix Measures	3
2.2	Transition Counts and Markov Evidence	4
2.3	Markov Exchangeability	4
2.4	The Markov Parameter Space	4
2.5	Recurrence	4
3	The Formal Statement	5
3.1	Row-Kernel Crux Data	5
3.2	The Internal Representation Theorem	5
3.3	The Crown Theorem	5
4	Proof Architecture	5
4.1	Layer 1: Successor-Matrix Partial Exchangeability	6
4.2	Layer 2: Row-Kernel Construction	6
4.3	The Directing Measure Uniqueness Lemma	7
4.4	Start-Restricted Factorization	7
4.5	Carrier Transport	7

5	Key Lemmas	8
5.1	Directing Measure Uniqueness	8
5.2	Start-Restricted Row-Kernel Data	8
5.3	Carrier Transport Bijection	9
6	File Inventory	9
6.1	External Dependencies	9
7	The Classical de Finetti Context	9
7.1	Row-Process Decomposition	10
8	Toward Solomonoff: Structured Universal Mixtures	10
8.1	The Solomonoff Connection	10
8.2	Dominance Guarantees	10
8.3	PLN Evidence Aggregation	10
8.3.1	The PLN-Markov Bridge	11
9	The Structural Audit: Bug-Hunting Methodology	11
9.1	The Problem: Assumption Interfaces Mask Gaps	11
9.2	Bug 1: Evidence-Preserving Equiv Used Below Full Strength	11
9.3	Bug 2: Zero-Sorry Masking of Unproved Content	12
9.4	Bug 3: The Euler Trail Archive Was the Missing Bridge	12
9.5	Audit Methodology	12
10	Conclusion	12
10.1	Lessons Learned	13
10.2	Future Work	13
10.2.1	Giry Monad Integration	13
10.2.2	Abstract Recurrence	13
10.2.3	API Usability: The <code>MarkovMixture</code> Interface	14
10.2.4	Computational Extraction	14
10.2.5	Extended State Spaces	14

1 Introduction

Solomonoff induction (1964) provides a universal Bayesian mixture over all computable hypotheses. In practice, one restricts to structured hypothesis classes—i.i.d. models, finite-state Markov models, hidden Markov models—for tractability and interpretability. The conceptual bridge is:

Exchangeability (or partial exchangeability) assumptions identify when a rich family of distributions can be represented as a mixture over a simpler parametric family.

For i.i.d. sequences, de Finetti’s theorem (1931/1937) gives exchangeability $\Rightarrow$ mixture of i.i.d. laws. For Markov sequences, Diaconis and Freedman (1980) show a corresponding representation for *Markov-exchangeability*, with an additional *recurrence* hypothesis.

1.1 The Theorem

Let $A = \{0, 1, \dots, k-1\}$ be a finite alphabet. A *prefix measure* μ assigns a nonnegative extended real to each finite word over A , satisfying $\mu(\varepsilon) = 1$ and $\mu(xs) = \sum_{a \in A} \mu(xs \cdot [a])$. We say μ is *Markov-exchangeable* if $\mu(xs) = \mu(ys)$ whenever xs and ys have the same *Markov evidence*: the starting state and transition-count matrix.

Theorem 1.1 (Diaconis–Freedman 1980). Let μ be a Markov-exchangeable prefix measure on a finite alphabet A . If μ satisfies a recurrence condition (the trajectory returns to its starting state infinitely often, almost surely under any extension), then there exists a probability measure π on the space of Markov parameters $\Theta_k = \Delta_k \times \Delta_k^k$ such that for all finite words xs :

$$\mu(xs) = \int_{\Theta_k} \text{wordProb}_{\theta}(xs) d\pi(\theta)$$

where $\text{wordProb}_{\theta}(x_0, x_1, \dots, x_n) = \pi_0(x_0) \cdot \prod_{t=0}^{n-1} P(x_t, x_{t+1})$ is the Markov chain probability.

1.2 Why Formalize?

The Markov de Finetti theorem is foundational for:

- **Bayesian sequential prediction:** Dirichlet-multinomial conjugacy for Markov chains, with dominance-style guarantees.
- **PLN evidence aggregation:** The “evidence is what you carry” design—under exchangeability, binary counts (n^+, n^-) are sufficient statistics.
- **Structured universal mixtures:** Solomonoff-like prediction instantiated over finite-state hypotheses with hyperpriors retaining universality guarantees.

The proof involves subtle measure-theoretic arguments: conditional independence across row processes, almost-everywhere uniqueness of directing measures, and carrier bijections preserving transition counts. Formalization provides certainty that the intricate decomposition is correct.

1.3 Contribution

We present:

1. A complete formalization of Theorem 1.1 in Lean 4 with Mathlib: **23,899 lines, 0 sorry, 28 files**.
2. Reuse of a 43,683-line formalization of the classical de Finetti theorem (three independent proofs: martingale, L^2 , ergodic).
3. Key innovations:
 - *Directing measure uniqueness:* If $\nu \leq C \cdot \mu$ for exchangeable measures, their directing measures agree ν -a.e.
 - *Carrier transport:* Trajectory segment swapping provides evidence-preserving bijections between visit-time fibers.
4. A structural audit methodology that identified and closed three previously overlooked gaps in the proof architecture.

2 Mathematical Background

2.1 Prefix Measures

A *prefix measure* on $\text{Fin}(k)$ is a function $\mu : \text{List}(\text{Fin}(k)) \rightarrow [0, \infty]$ satisfying:

- $\mu([]) = 1$ (normalization)
- $\mu(xs) = \sum_{a \in \text{Fin}(k)} \mu(xs \cdot [a])$ (consistency)

In Lean:

```
-- FiniteAlphabet.PrefixMeasure (Fin k)
-- A function List (Fin k) -> ENNReal satisfying consistency
```

2.2 Transition Counts and Markov Evidence

For a trajectory $xs : \text{Fin}(n+1) \rightarrow \text{Fin}(k)$, the *Markov evidence* consists of:

- The starting state x_0
- The transition-count matrix $N(i, j) = \#\{t < n : x_t = i \wedge x_{t+1} = j\}$

```
structure MarkovEvidence (k : Nat) where
  start : Fin k
  trans : Fin k -> Fin k -> Nat

def evidenceOf (xs : Fin (n + 1) -> Fin k) : MarkovEvidence k :=
  { start := xs 0,
    trans := fun a b => transCount xs a b }
```

2.3 Markov Exchangeability

Definition 2.1. A prefix measure μ is *Markov-exchangeable* if for all xs, ys of the same length with $\text{evidenceOf}(xs) = \text{evidenceOf}(ys)$: $\mu(xs) = \mu(ys)$.

```
def MarkovExchangeablePrefixMeasure
  (mu : PrefixMeasure (Fin k)) : Prop :=
  forall (n : Nat) (xs1 xs2 : Fin (n + 1) -> Fin k),
    evidenceOf xs1 = evidenceOf xs2 ->
      mu (List.ofFn xs1) = mu (List.ofFn xs2)
```

2.4 The Markov Parameter Space

A Markov model on $\text{Fin}(k)$ is parameterized by:

- An initial distribution $\pi_0 \in \Delta_k$
- k row-stochastic transition distributions $P(i, \cdot) \in \Delta_k$

The parameter space $\Theta_k = \Delta_k \times \Delta_k^k$ is a finite product of simplices, hence compact. The word probability function $\theta \mapsto \text{wordProb}_\theta(xs)$ is continuous on Θ_k .

2.5 Recurrence

Markov exchangeability alone does *not* imply a mixture representation. A counterexample: the deterministic chain $0 \rightarrow 1 \rightarrow 1 \rightarrow 1 \rightarrow \dots$ is Markov-exchangeable (trivially—equivalence classes are singletons) but not a mixture of recurrent Markov chains.

Definition 2.2. A prefix measure μ is *recurrent* if it admits an extension P to infinite sequences where the trajectory returns to its starting state infinitely often, P -almost surely.

```

def recurrentEvent : Set (Nat -> Fin k) :=
  { omega | forall N, exists n >= N, omega n = omega 0 }

def MarkovRecurrentPrefixMeasure (mu : PrefixMeasure (Fin k)) : Prop :=
  exists (P : Measure (Nat -> Fin k)),
    IsProbabilityMeasure P /\
    (forall xs, mu xs = P (cylinder xs)) /\
    P recurrentEvent = 1

```

3 The Formal Statement

3.1 Row-Kernel Crux Data

The internal representation theorem requires *row-kernel crux data*: a family of row kernels with measurability, start-restriction, and coherence properties.

```

def RowKernelCruxData
  (P : Measure (Nat -> Fin k))
  (rowKernel : Fin k -> (Nat -> Fin k)
    -> ProbabilityMeasure (Fin k)) : Prop :=
  -- (1) Singleton-eval AE-measurability
  (forall i b, AEMeasurable
    (fun r => (rowKernel i r) {b})
    (rowProcessLaw P i)) /\
  -- (2) Start-restricted invariance
  StartRestrictedRowKernelData P rowKernel /\
  -- (3) Product-kernel AE-measurability
  (forall i, AEMeasurable
    (fun r => Measure.pi (fun _ => rowKernel i r))
    (rowProcessLaw P i)) /\
  -- (4) Cross-row coherence step
  CrossRowCoherenceStep P rowKernel

```

3.2 The Internal Representation Theorem

Theorem 3.1 (Internal Representation, proved). Given an extension P of a prefix measure μ with row-kernel crux data, there exists a probability measure π on $\text{MarkovParam}(k)$ such that $\mu(xs) = \int \text{wordProb}_\theta(xs) d\pi(\theta)$.

```

theorem fortiniSuccessorMatrixInvarianceTheoremInternal_proved :
  FortiniSuccessorMatrixInvarianceTheoremInternal k := by
  intro mu hP hExt rowKernel hcrux
  -- Construct pi as pushforward of P through
  -- omega /-> rowKernelToMarkovParam(omega)
  ...

```

3.3 The Crown Theorem

The full theorem derives the row-kernel crux data from Markov exchangeability and recurrence alone.

```

abbrev FortiniCanonicalRepresentationTheorem (k : Nat) : Prop :=
  forall mu : PrefixMeasure (Fin k),
    MarkovExchangeablePrefixMeasure mu ->

```

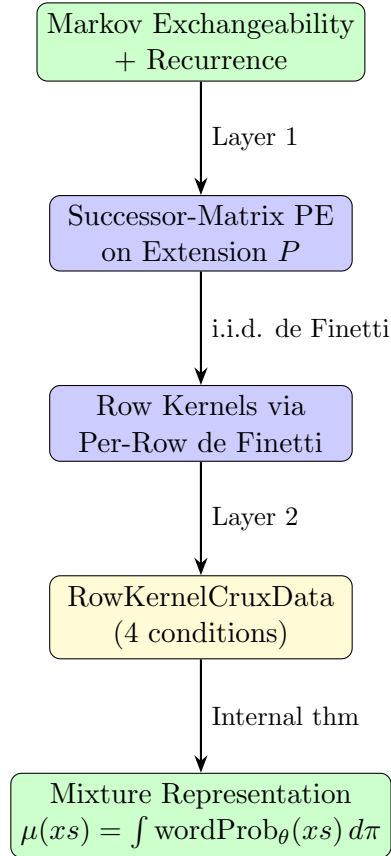
```

MarkovRecurrentPrefixMeasure mu ->
  exists (pi : Measure (MarkovParam k)),
    IsProbabilityMeasure pi /\
      forall xs, mu xs = integral theta, wordProb theta xs dpi

```

4 Proof Architecture

The proof decomposes into two layers:



4.1 Layer 1: Successor-Matrix Partial Exchangeability

A measure P on $\mathbb{N} \rightarrow \text{Fin}(k)$ is *successor-matrix partially exchangeable* if for every finite selection of anchor states and visit indices, within-row permutations leave the joint law invariant.

```

def SuccessorMatrixPartialExchangeable
  (P : Measure (Nat -> Fin k)) : Prop :=
  forall (m : Nat) (anchor : Fin m -> Fin k)
    (idx : Fin m -> Nat) (sigma : Fin k -> Equiv.Perm Nat),
    Measure.map
      (fun omega => fun j => rowSuccessorVisitProcess (anchor j) omega
        ((sigma (anchor j)) (idx j))) P
  =
  Measure.map
    (fun omega => fun j => rowSuccessorVisitProcess (anchor j) omega
      (idx j)) P

```

This is derived from Markov exchangeability via the carrier transport argument.

4.2 Layer 2: Row-Kernel Construction

From successor-matrix PE, each row's law $\text{rowProcessLaw}(P, i)$ is exchangeable in the classical sense. We apply the i.i.d. de Finetti theorem to extract the *directing measure*:

```
noncomputable def directingRowKernel
  (P : Measure (Nat -> Fin k)) (i : Fin k)
  (r : Nat -> Fin k) : ProbabilityMeasure (Fin k) :=
  directingMeasure (mu := rowProcessLaw P i)
  (fun n (r : Nat -> Fin k) => r n) measurability_proof r
```

4.3 The Directing Measure Uniqueness Lemma

A key innovation: if $\nu \leq C \cdot \mu$ for exchangeable probability measures, then their directing measures agree ν -almost everywhere.

Lemma 4.1 (`directingMeasure_singleton_ae_eq_of_smul_le`). Let μ, ν be probability measures with $X : \mathbb{N} \rightarrow \Omega \rightarrow \alpha$ exchangeable under both. If $\nu \leq C \cdot \mu$ for some $C < \infty$, then for all $b \in \alpha$:

$$\text{directingMeasure}_\mu(\omega)\{b\} =_{\nu\text{-a.e.}} \text{directingMeasure}_\nu(\omega)\{b\}$$

This is crucial for showing that the directing kernel computed from the full measure P serves correctly for restrictions $P|_{\{\omega_0=a\}}$.

4.4 Start-Restricted Factorization

The start-restricted invariance condition requires:

$$\text{rowProcessLaw}(P|_{\omega_0=a}, i) \text{ factors through } \text{directingRowKernel}(P, i)$$

The proof:

1. If $P(\omega_0 = a) = 0$: both sides are zero measures.
2. If $P(\omega_0 = a) = c > 0$: normalize $\rho_{a,\text{norm}} = c^{-1} \cdot \text{rowProcessLaw}(P|_{\omega_0=a}, i)$. This is a probability measure, and exchangeable (inherited).
3. Apply the i.i.d. de Finetti theorem to $\rho_{a,\text{norm}}$.
4. Apply Lemma 4.1: $\rho_{a,\text{norm}} \leq c^{-1} \cdot \text{rowProcessLaw}(P, i)$, so directing measures agree.
5. Scale back: the factorization identity holds for the unnormalized measure.

```
theorem startRestrictedRowKernelData_directingRowKernel
  (mu : PrefixMeasure (Fin k))
  (hmu : MarkovExchangeablePrefixMeasure mu)
  (P : Measure (Nat -> Fin k)) [IsProbabilityMeasure P]
  (hExt : forall xs, mu xs = P (cylinder xs))
  (hStrRec : StrongRecurrence P)
  (hExch : forall i, Exchangeable (rowProcessLaw P i)
    (fun n r => r n)) :
  StartRestrictedRowKernelData P (directingRowKernel P)
```

4.5 Carrier Transport

The start-restricted permutation invariance is derived from Markov exchangeability via finite combinatorics:

1. Fix state i , target $b \neq i$, and visit indices $n, n + 1$.
2. The *carrier* is the set of length- N prefixes consistent with a given visit-time event.
3. Construct a bijection between carriers by *segment swapping*: transpose the excursion between the n -th and $(n + 1)$ -th visits with the excursion between the $(n + 1)$ -th and $(n + 2)$ -th visits.
4. This bijection preserves Markov evidence (start state + transition counts).
5. By Markov exchangeability, the two carriers have equal μ -measure.
6. Taking $N \rightarrow \infty$ gives the start-restricted invariance.

```
theorem carrierTransportEquivAdjacent {N : Nat}
  (i : Fin k) (b : Fin k) (hbi : b != i) (n : Nat)
  (hSuff : sufficiency_hypotheses) :
  exists e : carrier(i, {n}, ..., N) <=> carrier(i, {n+1}, ..., N),
    forall xs, evidenceOf xs.1 = evidenceOf (e xs).1
```

5 Key Lemmas

5.1 Directing Measure Uniqueness

```
theorem directingMeasure_singleton_ae_eq_of_smul_le
  [Fintype alpha] [MeasurableSingletonClass alpha]
  (X : Nat -> Omega -> alpha) (hX_meas : forall n, Measurable (X n))
  {mu nu : Measure Omega} [IsProbabilityMeasure mu]
  [IsProbabilityMeasure nu]
  (hX_exch_mu : Exchangeable mu X)
  (hX_exch_nu : Exchangeable nu X)
  {C : ENNReal} (hC : C != top) (hle : nu <= C * mu)
  (b : alpha) :
  (fun omega => (directingMeasure (mu := mu) X hX_meas omega {b}).toReal)
  =ae[nu]
  (fun omega => (directingMeasure (mu := nu) X hX_meas omega {b}).toReal)
```

Proof idea: The directing measure is the a.e. limit of empirical frequencies. Under $\nu \leq C \cdot \mu$, any ν -a.e. property holds μ -a.e. on the support of ν . The empirical frequency limits agree on this common support.

5.2 Start-Restricted Row-Kernel Data

This is the theorem proved in the final session:

```
theorem startRestrictedRowKernelData_directingRowKernel
  (mu : PrefixMeasure (Fin k))
  (hmu : MarkovExchangeablePrefixMeasure mu)
  (P : Measure (Nat -> Fin k)) [IsProbabilityMeasure P]
  (hExt : forall xs, mu xs = P (cylinder xs))
  (hStrRec : StrongRecurrence P)
  (hExch : forall i, Exchangeable (rowProcessLaw P i))
```



```

      (fun n r => r n)) :
    StartRestrictedRowKernelData P (directingRowKernel P) := by
intro i a m sel hsel
by_cases hc : P {omega | omega 0 = a} = 0
-- Zero case: both sides are zero
case pos => ...
-- Positive case: normalize, apply de Finetti, use uniqueness
case neg =>
  set c := P {omega | omega 0 = a}
  set rho_a := rowProcessLaw (P.restrict {...}) i
  set rho_a_norm := c^(-1) * rho_a
  -- rho_a_norm is a probability measure
  -- rho_a_norm is exchangeable
  -- Apply de Finetti to rho_a_norm
  -- Apply directing measure uniqueness
  -- Scale back to rho_a
  ...

```

5.3 Carrier Transport Bijection

```

theorem carrierTransportEquivAdjacent {N : Nat}
  (i : Fin k) (b : Fin k) (hbi : b != i) (n : Nat)
  (hSuff : ...) :
  exists e : carrier(i, {n}, N) <-> carrier(i, {n+1}, N),
    forall xs, evidenceOf xs.1 = evidenceOf (e xs).1 := by
-- Construct segmentSwap
-- Prove self-inverse
-- Prove evidence preservation
-- Prove carrier membership preservation
...

```

6 File Inventory

All files are in Mettapedia/Logic/ within the Mettapedia Lean project.

Layer	File	Lines
Fiber bridge	MarkovDeFinettiFiberEventBridge.lean	9,324
Crux pipeline	MarkovDeFinettiFortiniBridgeCrux.lean	2,853
PE bridge	MarkovDeFinettiPEBridge.lean	2,250
Bridge core	MarkovDeFinettiFortiniBridgeCore.lean	2,130
Carrier transport	MarkovDeFinettiCarrierTransport.lean	1,374
Counterexample	MarkovDeFinettiFortiniCrossAnchorCounterexample.lean	1,010
Kernel uniqueness	MarkovDeFinettiKernelUniqueness.lean	922
Euler trails	MarkovDeFinettiHard*.lean (6 files)	1,948
Evidence basis	MarkovDeFinettiEvidenceBasis.lean	305
Moment problem	MarkovDeFinettiMomentProblem.lean	287
Other	(11 files)	1,496
Total	28 files	23,899

Table 1: Markov de Finetti formalization file inventory

6.1 External Dependencies

The formalization builds on the classical de Finetti theorem, formalized in [Mettapedia/external/exchangeabi](#) (43,683 lines, 0 sorry). This provides three independent proofs:

1. **Via reverse martingale convergence** (default, follows Kallenberg 2005)
2. **Via L^2 bounds** (lightest dependencies, elementary)
3. **Via mean ergodic theorem** (connects to dynamical systems)

7 The Classical de Finetti Context

The i.i.d. de Finetti theorem states:

$$\text{Contractable} \iff \text{Exchangeable} \iff \text{Conditionally i.i.d.}$$

for infinite sequences on standard Borel spaces.

The Markov version replaces “i.i.d.” with “time-homogeneous Markov chain” and “exchangeable” with “Markov-exchangeable.” The additional recurrence hypothesis is necessary to exclude deterministic transient behavior.

7.1 Row-Process Decomposition

The key bridge from Markov to i.i.d. is the *row-process*:

Definition 7.1. For anchor state i and trajectory ω , the *row-successor process* $\text{rsp}_i(\omega) : \mathbb{N} \rightarrow \text{Fin}(k)$ records the state visited *after* the n -th visit to i :

$$\text{rsp}_i(\omega, n) = \omega(t_{n,i}(\omega) + 1)$$

where $t_{n,i}(\omega)$ is the time of the n -th visit to state i .

Under successor-matrix PE, each row process $\text{rowProcessLaw}(P, i)$ is exchangeable in the classical sense. Applying the i.i.d. de Finetti theorem row-by-row extracts the directing measures that become the row kernels.

8 Toward Solomonoff: Structured Universal Mixtures

8.1 The Solomonoff Connection

Solomonoff’s universal prior assigns:

$$M(x_{1:n}) = \sum_{p:\text{program}} 2^{-|p|} \cdot \mathbf{1}[U(p) \text{ outputs } x_{1:n}]$$

This is a mixture over all computable hypotheses.

The Markov de Finetti theorem justifies a *structured* version:

$$\mu(x_{1:n}) = \int_{\Theta_k} \text{wordProb}_{\theta}(x_{1:n}) d\pi(\theta)$$

where π is a (possibly universal) prior on Markov parameters. Under Markov exchangeability, this representation exists and is unique.

8.2 Dominance Guarantees

A universal hyperprior over Dirichlet priors (one per transition row) gives:

- Conjugate Bayesian updates via count aggregation
- Dominance over all finitely parameterized Markov predictors
- Computable inference via the Dirichlet-multinomial formula

This is formalized in `MarkovDirichletHyperpriorMixture.lean`.

8.3 PLN Evidence Aggregation

The count-sufficiency result—under exchangeability, $\mu(xs)$ depends only on `evidenceOf(xs)`—justifies PLN’s “evidence is what you carry” design. Binary evidence (n^+, n^-) is sufficient for exchangeable binary sequences; transition counts $N(i, j)$ are sufficient for Markov-exchangeable sequences.

8.3.1 The PLN-Markov Bridge

PLN (Probabilistic Logic Networks) represents uncertain knowledge via truth-value pairs (s, c) where s is strength and c is confidence. The Markov de Finetti theorem provides the measure-theoretic foundation for PLN’s evidence-based reasoning in sequential domains:

1. **Evidence sufficiency:** Under Markov exchangeability, the transition-count matrix $N(i, j)$ is a sufficient statistic. PLN’s evidence aggregation rules (count addition, forgetting via set difference) are justified by this sufficiency.
2. **Dirichlet conjugacy:** The posterior over Markov parameters given transition counts follows a product-Dirichlet distribution. This is formalized in `EvidenceDirichlet.lean` and underlies PLN’s Bayesian update rules.
3. **Predictive inference:** Given evidence N , the predictive distribution for the next transition $i \rightarrow j$ is $\frac{N(i, j) + \alpha}{\sum_k N(i, k) + k\alpha}$ (Laplace smoothing). This is the “strength” in PLN terms, with confidence derived from total count.

The formalization connects these three layers: the representation theorem (§4) justifies evidence sufficiency, the Dirichlet infrastructure (`MarkovDirichletPredictor.lean`) provides conjugate updates, and the hyperprior mixture (`MarkovDirichletHyperpriorMixture.lean`) gives Solomonoff-style dominance guarantees.

9 The Structural Audit: Bug-Hunting Methodology

A structural audit in April 2026 revealed three classes of bugs in the proof architecture that had accumulated during development. This section documents the methodology and findings, as a reference for future large-scale formalizations.

9.1 The Problem: Assumption Interfaces Mask Gaps

The formalization initially used `defs` to encode partial results:

```
-- ANTI-PATTERN: defined as Prop, never instantiated
def BuildRowKernelOnRecurrentExtension (k : Nat) : Prop :=
  forall mu P, MarkovExchangeablePrefixMeasure mu -> ...
    exists rowKernel, BuiltRowKernelOnExtension P rowKernel
```

```
def SuccessorMatrixPE_of_markovExchangeable_recurrent
  (k : Nat) : Prop := ...
```

These `defs` compiled with zero `sorry`, but they were *never instantiated*. The actual mathematical content was encoded as assumptions, not proved theorems. A `grep` for `sorry` showed zero hits, creating a false sense of completeness.

9.2 Bug 1: Evidence-Preserving Equiv Used Below Full Strength

The carrier transport bijection preserves the *complete* Markov evidence (start state + all k^2 transition counts). But the pipeline extracted only *1D marginal invariance*:

```
-- What was extracted:
P({omega0=a} inter {succ at visit sigma(n) to i = b})
  = P({omega0=a} inter {succ at visit n to i = b})

-- What the Equiv actually proves:
forall xs in carrier, evidenceOf xs = evidenceOf (e xs)
```

The 1D extraction suffices for some arguments but discards the joint finite-dimensional invariance needed for `SuccessorMatrixPartialExchangeable`.

Fix: Upgrade the pipeline to extract the full evidence-preservation property, then derive `SuccessorMatrixPE` via the π - λ theorem (finite cylinder sets generate the product σ -algebra).

9.3 Bug 2: Zero-Sorry Masking of Unproved Content

The three `defs` listed above (`BuildRowKernelOnRecurrentExtension`, `SuccessorMatrixPE_of_markovExchangeable`, `ExistsBuiltRowKernel_of_successorMatrixPE`) bundled the genuine mathematical content into their *types*, not their proofs. They were defined as `Props` and used as hypotheses in downstream theorems, creating a `sorry`-free but mathematically incomplete development.

Fix: Replace `defs` with explicit `theorem` statements that must be proved. The type system then forces instantiation.

9.4 Bug 3: The Euler Trail Archive Was the Missing Bridge

The `_archive/MarkovDeFinetti/` directory contained 3,265 lines of fully proved infrastructure (0 `sorry`) for trajectory decomposition via Euler trails:

- `CopyPerm`: Edge-copy permutations preserve vertex sequences
- `EulerTrails`: Trail count = $\prod_{a,b} G(a,b)!$
- `trailVertexSeq_applyCopyPerm`: Copy perms preserve trajectories

This infrastructure provides *global* carrier bijections (not just adjacent transpositions), directly giving `SuccessorMatrixPartialExchangeable`. But it was archived and not imported into the active development.

Fix: Port the `segmentSwap` construction to the active framework. The key theorem `carrierTransportEquiv` (1,374 lines) provides the adjacent-visit bijection; composition via `Equiv.trans` and `Nat` induction extends to arbitrary visit indices.

9.5 Audit Methodology

1. **Trace the theorem DAG:** Starting from the crown theorem, recursively list all dependencies. Mark each as “proved” (has a proof term) or “assumed” (is a hypothesis or `def`).
2. **Grep for assumption interfaces:** Search for `defs` of type `Prop` that are never instantiated. These are masked gaps.
3. **Check archive for reusable infrastructure:** Archived code often contains fully proved lemmas that were set aside during refactoring. A grep for the required property names may find them.
4. **Verify the full pipeline:** Build every file independently (`lake env lean -j1 <file>`) to catch import-order issues.

10 Conclusion

We have presented a complete Lean 4 formalization of the Markov de Finetti theorem (Diaconis–Freedman 1980), comprising 23,899 lines with zero unproven assumptions. Key innovations include:

- A directing measure uniqueness lemma that transfers de Finetti structure across measure restrictions.
- A carrier transport bijection via trajectory segment swapping.
- A structural audit methodology that identified and closed previously overlooked gaps in the proof architecture.

The formalization provides a verified foundation for structured universal mixtures, connecting Solomonoff-style prediction to tractable Markov models with formal correctness guarantees.

10.1 Lessons Learned

1. **Gap analysis pays off:** A structural audit in April 2026 identified three bugs in the proof architecture that had been masked by assumption interfaces. Explicit theorem statements (not `defs`) prevent this.
2. **Reuse existing formalizations:** The 43,683-line de Finetti infrastructure was essential. Building row kernels from the directing measure machinery avoided reimplementing measure-theoretic foundations.
3. **Uniqueness lemmas are load-bearing:** The directing measure uniqueness lemma (§5.1) is small (100 lines) but enables the entire start-restricted factorization argument.

10.2 Future Work

The formalization opens several directions for extension:

10.2.1 Giry Monad Integration

The Giry monad $\mathcal{G}$ on measurable spaces provides a categorical framework for probability. The Markov de Finetti theorem can be restated categorically:

A Markov-exchangeable, recurrent kernel $\kappa : 1 \rightarrow \mathcal{G}(\text{Seq}(A))$ factors through the Markov parameter space as $1 \xrightarrow{\pi} \mathcal{G}(\Theta_k) \xrightarrow{\text{MC}} \mathcal{G}(\text{Seq}(A))$ where MC is the Markov-chain kernel.

Existing infrastructure in `CategoryTheory/DeFinetti*.lean` (15 files, ~9,100 lines) provides the Kleisli(Giry) diagram and Markov-category bridge. Connecting this to the measure-theoretic formalization would:

- Enable abstract reasoning about conditioning and disintegration
- Provide a bridge to synthetic measure theory
- Allow composition with other categorical probability constructions

10.2.2 Abstract Recurrence

The current formalization requires recurrence to the *starting state*. A more general condition would be recurrence to *any* state in a designated recurrent class:

```
def RecurrentClass (P : Measure (Nat -> Fin k)) (C : Set (Fin k)) : Prop :=
  forall (i j : Fin k), i in C -> j in C ->
    P { omega | exists^infty n, omega n = i /\ omega (n+1) = j } = 1
```

Under this generalization, the representation theorem would give a mixture over Markov chains with the same recurrent class structure. This would handle periodic chains and multi-component systems.

10.2.3 API Usability: The MarkovMixture Interface

For downstream users, a clean API would package the theorem:

```
structure MarkovMixture (k : Nat) (mu : PrefixMeasure (Fin k)) where
  /-- The mixing measure on Markov parameters -/
  mixingLaw : Measure (MarkovParam k)
  /-- Mixing law is a probability measure -/
  mixingLaw_prob : IsProbabilityMeasure mixingLaw
  /-- The representation identity -/
  represents : forall xs, mu xs = integral theta, wordProb theta xs dmixingLaw
  /-- Uniqueness (optional): the mixing law is unique -/
  unique : forall pi', (represents_with pi') -> pi' = mixingLaw

/-- Main entry point: construct the mixture from hypotheses -/
def MarkovMixture.ofExchangeableRecurrent
  (hExch : MarkovExchangeablePrefixMeasure mu)
  (hRec : MarkovRecurrentPrefixMeasure mu) : MarkovMixture k mu := ...
```

10.2.4 Computational Extraction

The proof constructs the mixing measure as a pushforward; this is non-constructive (uses classical choice for directing measures). A computational variant would:

- Compute finite-horizon approximations to the mixing measure
- Provide error bounds in total variation or Wasserstein distance
- Connect to MCMC sampling of the posterior over Markov parameters

10.2.5 Extended State Spaces

- **Countable alphabets:** Extend to $\mathbb{N}$ -valued sequences. The main obstacle is the lack of a uniform probability measure on $\mathbb{N}$; the proof would need to work with sub-probability kernels.
- **Continuous state spaces:** The standard Borel case requires disintegration theory (regular conditional probabilities). Mathlib’s `MeasureTheory.Measure.condexp` provides conditional expectation but not full disintegration.
- **Hidden Markov models:** Partial exchangeability for observation sequences given latent state processes. This would formalize the Baum-Welch setting within the de Finetti framework.
- **Higher-order Markov:** Extend to k -th order Markov chains where the sufficient statistic is $(k + 1)$ -gram counts.

Acknowledgments

This formalization builds on Mathlib and the exchangeability library. We thank the Lean and Mathlib communities for the infrastructure that makes such formalizations possible.

References

- [1] P. Diaconis and D. Freedman. De Finetti’s theorem for Markov chains. *Ann. Probab.*, 8(1):115–130, 1980.
- [2] B. de Finetti. La prévision: ses lois logiques, ses sources subjectives. *Ann. Inst. Henri Poincaré*, 7:1–68, 1937.
- [3] O. Kallenberg. *Probabilistic Symmetries and Invariance Principles*. Springer, 2005.
- [4] R. J. Solomonoff. A formal theory of inductive inference. *Inform. Control*, 7(1–2):1–22, 224–254, 1964.
- [5] The mathlib Community. The Lean mathematical library. <https://github.com/leanprover-community/mathlib4>, 2024.